

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 1 | SLO 1: Classes & Objects (Code: 2011)

Q: Activity: Create a Product class with the following attributes: productName (String), price (double), quantity (int). The class should have methods to display product details. Create two objects for the Product class and display the respective product details.

Ans: Java Implementation:

```
class Product {
    String productName;
    double price;
    int quantity;

    Product(String name, double p, int q) {
        this.productName = name;
        this.price = p;
        this.quantity = q;
    }

    void displayDetails() {
        System.out.println("Product Name: " + productName);
        System.out.println("Price: $" + price);
        System.out.println("Quantity: " + quantity);
        System.out.println("Total Value: $" + (price * quantity));
        System.out.println("-----");
    }
}

public class ProductDemo {
    public static void main(String[] args) {
        Product p1 = new Product("Laptop", 1200.50, 5);
        Product p2 = new Product("Wireless Mouse", 25.75, 20);
        p1.displayDetails();
        p2.displayDetails();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 1 | SLO 2: Constructors, this Keyword & Methods (Code: 2012)

Q: Activity: Design a Box class that implements Default, Parameterized, and Copy Constructors. Include a method to compute volume and demonstrate the use of the 'this' keyword to resolve shadowing.

Ans: Java Implementation:

```
class Box {
    double width, height, depth;

    // 1. Default Constructor
    Box() {
        this.width = 1.0;
        this.height = 1.0;
        this.depth = 1.0;
    }

    // 2. Parameterized Constructor with 'this' reference
    Box(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }

    // 3. Copy Constructor
    Box(Box other) {
        this.width = other.width;
        this.height = other.height;
        this.depth = other.depth;
    }

    double volume() {
        return width * height * depth;
    }
}

public class BoxDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(10.0, 5.0, 2.0);
        Box b3 = new Box(b2);

        System.out.println("Default Box Volume: " + b1.volume());
        System.out.println("Parameterized Box Volume: " + b2.volume());
        System.out.println("Copied Box Volume: " + b3.volume());
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 2 | SLO 1: Data Types, Variables & Scope (Code: 2021)

Q: Activity: Implement a Java program demonstrating primitive data types, variable scope, dynamic initialization, and the distinction between local, instance, and static variables.

Ans: Java Implementation:

```
public class VariableScopeDemo {
    // Static (class) variable - single copy per class
    static int classCounter = 0;

    // Instance variable - unique per object
    String objectLabel;

    public VariableScopeDemo(String label) {
        this.objectLabel = label;
        classCounter++;
    }

    public void compute() {
        // Local variable - exists only in method scope
        byte byteVal = 100;
        double dynamicVal = Math.sqrt(byteVal);
        System.out.println("Label: " + objectLabel);
        System.out.println("Dynamic Local Value: " + dynamicVal);
        System.out.println("Class Counter (Static): " + classCounter);
    }

    public static void main(String[] args) {
        VariableScopeDemo d1 = new VariableScopeDemo("Object Alpha");
        d1.compute();
        VariableScopeDemo d2 = new VariableScopeDemo("Object Beta");
        d2.compute();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 2 | SLO 2: Identifiers, Operators & Access Modifiers (Code: 2022)

Q: Activity: Demonstrate Java access modifiers (private, default, protected, public) and operator precedence with arithmetic, relational, and logical operators in an application.

Ans: Java Implementation:

```
class AccessDemo {
    private int secretKey = 999;    // Accessible only within class
    int defaultVal = 10;           // Package-private
    protected double protectedVal = 55.5; // Subclasses & package
    public String publicMsg = "Open";    // Accessible anywhere

    public int getSecretKey() {
        return secretKey; // Controlled accessor
    }
}

public class OperatorPrecedenceDemo {
    public static void main(String[] args) {
        AccessDemo demo = new AccessDemo();
        System.out.println("Public: " + demo.publicMsg);
        System.out.println("Secret via getter: " + demo.getSecretKey());

        // Operator precedence evaluation
        int a = 10, b = 5, c = 2;
        int res = a + b * c; // Multiplication precedes addition
        boolean check = (a > b) && (b != 0);
        System.out.println("Result of 10 + 5 * 2 = " + res);
        System.out.println("Logical condition (10 > 5 && 5 != 0): " + check);
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 3 | SLO 1: Inheritance & Method Overriding (Code: 2031)

Q: Activity: Implement Single and Multilevel Inheritance representing an Employee hierarchy. Create a superclass Employee and subclasses Manager and Executive. Demonstrate super constructor invocation and method overriding.

Ans: Java Implementation:

```
class Employee {
    String name;
    double basicSalary;

    Employee(String name, double basicSalary) {
        this.name = name;
        this.basicSalary = basicSalary;
    }

    double calculateSalary() {
        return basicSalary;
    }
}

class Manager extends Employee {
    double bonus;

    Manager(String name, double basicSalary, double bonus) {
        super(name, basicSalary); // Super constructor
        this.bonus = bonus;
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + bonus;
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Employee emp = new Employee("Alice", 50000);
        Manager mgr = new Manager("Bob", 70000, 15000);
        System.out.println("Employee Salary: " + emp.calculateSalary());
        System.out.println("Manager Salary: " + mgr.calculateSalary());
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 3 | SLO 2: Polymorphism (Compile-time & Runtime) (Code: 2032)

Q: Activity: Write a program demonstrating both Compile-Time Polymorphism (Method Overloading) and Runtime Polymorphism (Method Overriding via Dynamic Method Dispatch) using geometric shapes.

Ans: Java Implementation:

```
class ShapeCalculator {
    // Method Overloading (Compile-time polymorphism)
    double area(double radius) {
        return Math.PI * radius * radius;
    }
    double area(double length, double width) {
        return length * width;
    }
}

class Shape {
    void draw() {
        System.out.println("Drawing generic shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        ShapeCalculator calc = new ShapeCalculator();
        System.out.println("Circle Area: " + calc.area(4.0));
        System.out.println("Rectangle Area: " + calc.area(4.0, 6.0));

        // Dynamic Method Dispatch (Runtime Polymorphism)
        Shape myShape = new Circle();
        myShape.draw();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 4 | SLO 1: Abstract Classes & Interfaces (Code: 2041)

Q: Activity: Design a banking system illustrating the difference between an Abstract Class and an Interface. Implement an abstract class BankAccount and an interface Transaction.

Ans: Java Implementation:

```
interface Transaction {
    void deposit(double amount);
    void withdraw(double amount);
}

abstract class BankAccount implements Transaction {
    String accountNumber;
    double balance;

    BankAccount(String accNo, double bal) {
        this.accountNumber = accNo;
        this.balance = bal;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: $" + amount + ", New Balance: $" + balance);
    }

    abstract void applyInterest();
}

class SavingsAccount extends BankAccount {
    SavingsAccount(String accNo, double bal) {
        super(accNo, bal);
    }

    public void withdraw(double amount) {
        if (balance >= amount) balance -= amount;
    }

    void applyInterest() {
        balance += balance * 0.04;
        System.out.println("Interest applied. Balance: $" + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        SavingsAccount sa = new SavingsAccount("SA-101", 1000.0);
        sa.deposit(500.0);
        sa.applyInterest();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 4 | SLO 2: Packages & Information Hiding (Code: 2042)

Q: Activity: Demonstrate the concept of package declaration, importing packages, and how access specifiers enforce encapsulation across package boundaries in Java.

Ans: Java Implementation:

```
// File: srm/cse/app/SecurityModule.java
package srm.cse.app;

public class SecurityModule {
    public void grantAccess() {
        System.out.println("Public access granted across packages.");
    }

    protected void internalConfig() {
        System.out.println("Protected config: available in subclasses/package.");
    }
}

// File: ClientApplication.java
import srm.cse.app.SecurityModule;

public class ClientApplication {
    public static void main(String[] args) {
        SecurityModule sec = new SecurityModule();
        sec.grantAccess(); // Accessible
        // sec.internalConfig(); // Compilation error: protected method outside package
    }
}
```


SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 5 | SLO 1: Exception Handling (try-catch-finally) (Code: 2051)

Q: Activity: Implement a robust exception handling program illustrating try, multiple catch blocks, and the finally block for resource cleanup.

Ans: Java Implementation:

```
public class ExceptionHandlingDemo {
    public static void main(String[] args) {
        try {
            int[] arr = {10, 20, 30};
            int numerator = 100;
            int denominator = 0;
            // Triggers ArithmeticException
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.err.println("Caught Math Exception: Division by zero!");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Caught Array Index Exception!");
        } finally {
            System.out.println("Finally block executed: Cleaned up resources.");
        }
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 5 | SLO 2: Custom User-Defined Exceptions (Code: 2052)

Q: Activity: Create a custom user-defined exception InvalidAgeException to validate candidate eligibility for voting. Use throw and throws keywords.

Ans: Java Implementation:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionDemo {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age " + age + " is invalid: Must be 18 or older.");
        }
        System.out.println("Age verified: Eligible to vote!");
    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (InvalidAgeException e) {
            System.err.println("Custom Exception Caught: " + e.getMessage());
        }

        try {
            validateAge(21);
        } catch (InvalidAgeException e) {
            System.err.println(e.getMessage());
        }
    }
}
```